

AI 리터러시 케어 에이전트 최종 기획 보고서

읽기 행동을 측정하고 문해력 성장을 관리하는
폐루프(Closed-Loop) 멀티 에이전트 시스템

— "AI가 글을 대신 읽어주는 서비스가 아니라, 사용자의 문해력 성장을 관리하는 서비스" —

팀명	AllDayHappyDay	참가 인원	5명
지원 부문	SW부문	소속	숙명여자대학교 SW중심 대학사업단
문서 버전	v2.0 (확장 반영 개정 안)	작성일	2026-07-05
문서 목적	팀 내부 공유용 최종 기획 — 차별화 전략.핵심 지표.아키텍처.크롬 확장.역할 별 할 일 확정		

본 문서는 v1.0 최종 확정안에 실제 구현하며 확장한 내용을 반영한 개정본입니다 — ① 제품 형태를 파일 업로드 없는 크롬 확장으로 확정, ② PDF도 pdf.js 자체 뷰어로 웹과 동일 측정, ③ 익명 UUID 온보딩·REST 이벤트 전송·비용 0 원
칙을 설계 결정(ADR)으로 명문화. 일정형 데일리 플랜은 시간 무관 "할 일 정리"로 대체했습니다.



Executive Summary & 목차

한 장 요약 / Table of Contents

우리가 만드는 것은 "요약기"가 아니라 "문해력 성장 관리 시스템"이다.

사용자의 **읽기 행동(과정)**과 **이해도(결과)**를 측정해 **Literacy Score**라는 단일 지표로 정량화하고, 실시간 개입(넛지·퀴즈·재구성)으로 성장을 유도하며, 누적 시계열로 **성장 곡선**을 추적하는 **페루프(측정 → 개입 → 추적)** 멀티 에이전트 시스템이다. 이것이 ChatGPT 단순 요약과의 결정적 차이이다.

-24점

한국 언어능력
(PIAAC 2012→2023)

6+1

에이전트
(전문 5 + 오케스트레이터 +
QA)

1지표

Literacy Score
전후 비교 그래프

웹+PDF

크롬 확장
업로드 없이 측정

목차

- 1 프로젝트 개요 & 문제 정의
- 2 핵심 차별화 전략 — GPT와 무엇이 다른가
- 3 서비스 포지셔닝 & 타겟 사용자
- 4 핵심 지표: Literacy Score 설계 (이 보고서의 심장)
- 5 시스템 아키텍처 — 계층형 멀티 에이전트
- 6 에이전트 상세 명세 (6종)
- 7 RAG 적용 범위 & 기술 스택 / LLM 라우팅
- 8 검증·평가 체계 (Harness + QA 2층 구조)
- 9 최종 산출물 & 크롬 확장 (웹 + PDF pdf.js 뷰어 · 온보딩 · 설계 결정)
- 10 개발 우선순위 & 할 일 정리 (역할별 · 시간 무관)
- 11 팀 역할 분담
- 12 심사 예상 질문 & 방어 스크립트
- 13 리스크 & 대응 / 부록

이번 개정(v2.0)에서 "확정"한 5가지

1. **차별화 프레임 고정**: "GPT는 텍스트를 처리하고, 우리는 사람을 관리한다" — 페루프 구조로 설명.
2. **Literacy Score 단일 지표 정의**: 이해도 × 집중도 × 난이도 보정 → 전후 비교 그래프가 데모의 핵심 화면.
3. **제품 형태 = 크롬 확장**: 파일 업로드 없이 크롬에서 읽는 모든 글(웹 + PDF)에 실시간 개입. on/off 백그라운드 상시 동작.
4. **PDF도 웹과 동일하게**: 크롬 기본 PDF뷰어(PDFium) 대신 pdf.js 자체 뷰어로 자동 전환해 DOM으로 되돌려 동일 트래커/오버레이 재사용.

5. **비용 0·P11 0 원칙:** 익명 기기 UUID(로그인 없음), 로컬 문서 처리(서버 저장 없음), REST 이벤트 전송, 무료 라이선스 번들만 사용.

1.1 한 줄 정의

사용자가 읽고 있는 텍스트를 AI가 분석·재구성하고, 읽기 행동 데이터를 실시간 분석해 맞춤 개입(강조·쉬운 설명·퀴즈·요약)을 수행하여 끝까지 읽고 이해하도록 돕는 개인 맞춤형 AI 리더서 코치.

1.2 문제 정의 — 왜 지금 필요한가

문제	근거 / 현황
문해력 저하	OECD PIAAC 기준 한국 언어능력 점수 273점(2012) → 249점(2023)으로 24점 하락 . 27개국 평균보다 13점 낮은 수준으로, 2012년 평균 대비 +4점이었던 것에서 역전됨.
숏폼 중독·집중력 저하	짧은 콘텐츠 중심 소비로 긴 글을 끝까지 읽고 이해하는 능력 약화. 학습·업무 수행 능력 저하로 연결.
ADHD·난독증 사각지대	집중 유지·해독·이해에 추가 지원 필요. 기존 도구는 난독증 지원/집중 관리/요약이 파편화 되어 통합 서비스가 공백. 대부분 영어 기반이라 한국어 환자용은 전무.
기존 AI의 한계	ChatGPT·Gemini·Claude는 텍스트를 대신 읽어 요약 할 뿐, 사용자가 실제로 읽었는지·이해했는지·향상됐는지는 관리하지 않음.

1.3 우리의 해결 방향

AI가 내용을 **대신 읽어주는** 것이 아니라, 사용자가 **스스로 읽고 이해하도록** 돕는다. 읽기 행동·집중 상태를 실시간 분석하고 상황별 개입을 제공하여 독해 경험을 개선하고, 누적 데이터로 **장기 문해력 성장**에 기여한다.

근거 연구: AI 주석 기반 텍스트 제시(LARF, CHI 2025)에서 AI 주석 텍스트를 읽은 참가자는 세부 내용 기억에서 기존 대비 **18.3% 높은 점수**를 기록, 읽기 이해도·만족도 모두 유의미하게 향상. 본 시스템은 이를 한국어 환경에서 실시간 구현.

⚠ 이번 기획의 최대 리스크

심사위원 질문 "ChatGPT에 PDF 넣고 요약시키면 끝 아닌가요?" — 이 한 방에 무너지면 안 된다. 답은 기능 나열이 아니라 구조(폐루프)로 한다.

2.1 핵심 프레임

GPT는 텍스트를 처리하고, 우리는 사람을 관리한다.

2.2 폐루프(Closed-Loop) 구조 — 차별화의 본질

① 측정 (Measure)

읽기 행동(스크롤·체류·이탈) + 이해도(퀴즈)로 인지부하·집중도 정량화

② 개입 (Intervene)

상태에 따라 강조·쉬운 설명·넋지·즉석 퀴즈로 재집중·이해 유도

③ 추적 (Track)

누적 시계열로 Literacy Score 성장 곡선 산출 + 프로필 갱신

③의 프로필이 다시 ①②의 개인화에 반영되는 순환 구조

2.3 ChatGPT vs 본 서비스

기준	ChatGPT / 일반 AI	본 서비스
실제로 읽었는지 측정	× 불가	✓ 인지부하(스크롤·체류·이탈) 실시간
이해했는지 검증	× 요약만 제공	✓ 난이도 보정 이해도 점수
향상됐는지 추적	× 세션 휘발	✓ 누적 시계열 → 성장 곡선
개인화	△ 매번 초기화	✓ Dynamic Literacy Profile 누적
신뢰성(환각)	△ 검증 없음	✓ RAG 그라운드링 + Faithfulness 평가
사용 방식	△ 붙여넣기·업로드 필요	✓ 크롬에서 <u>보던 대로</u> 자동 측정(업로드 0)
목적	정보를 대신 처리	사용자가 스스로 성장

한 줄 방어: "ChatGPT는 한 번 읽고 잊지만, 우리 시스템은 사용자가 어떻게 읽는지 측정하고, 이해했는지 검증하고, 나아지는지 평생 추적합니다."

3.1 포지셔닝 결정

후보	채택	판단
AI Reading Coach	보류	"코치"는 챗봇처럼 들려 차별화가 약함.
Cognitive Reading OS	미채택	"운영체제"는 과장. 심사위원이 개념을 되물어 설명에 시간을 소모할 위험.
개인화 문해력 성장 관리 시스템	✓ 채택	"한 번 쓰고 끝"이 아니라 지속 관리·성장 파트너 임을 정확히 전달. 방어 용이.

최종 포지셔닝 문구: "읽기 행동을 측정해 문해력 성장을 지속적으로 관리하는 개인 맞춤형 성장 파트너"

3.2 타겟 사용자

세그먼트	니즈	핵심 가치
2030 청년 (1차)	숏폼 중독으로 긴 글 집중·완독이 어려움	집중도 회복 네티지 + 완독 게이미피케이션
학습자·취준생	논문·보고서·기사 등 고난도 텍스트 독해	난이도 재구성 + 이해도 검증
ADHD·난독증 (배리어프리)	집중 유지·해독·이해에 추가 지원 필요	한국어 AI 주석·청킹·용어풀이 (공백 시장)

3.3 시장 공백

난독증 폰트·집중 관리·요약 도구는 개별 존재하나 **통합 서비스는 공백**이며, 대부분 영어 기반. **한국어 기반 통합 문해력 성장 관리**는 사실상 전무 — 본 프로젝트의 진입 지점.

진입 전략: 무거운 앱 설치·업로드 대신 **크롬 확장**으로 마찰을 0에 가깝게 낮춰, "평소 읽던 뉴스·논문 위에서 바로 켜본다"는 낮은 진입장벽으로 첫 사용을 유도한다.

차별화를 눈에 보이게 만드는 유일한 장치. "문해력이 향상됐다"는 주장을 단일 정량 지표 + 전후 비교 그래프로 증명한다.

4.1 지표 구성

Literacy Score = f(이해도, 집중도, 난이도 보정)

- ① **Comprehension** (결과지표) = 퀴즈 정답률 ÷ 지문 난이도(한국어 가독성 지수)로 정규화
- ② **Engagement** (과정지표) = 스크롤 속도·체류시간·이탈률 → 능동읽기(Deep) vs 훑기(Skim) 점수
- ③ **Cross-Validation 보정** = 정답이어도 스크롤이 비정상적으로 빠르면(찍기 의심) 감점
- ④ **Longitudinal** = 단일 세션이 아닌 시계열 추세(Trend)로 성장 판정

→ 점수 엔진은 LLM이 아닌 순수 함수(결정적·재현성)로 구현한다. 같은 입력이면 같은 점수 = 심사 시연·검증에 유리.

4.2 두 가지 측정 차원

차원	유형	측정 방식
실시간 집중력	Process Metric	인지부하(Cognitive Load): 스크롤 속도·문단 체류 시간·페이지 이탈률 → 능동/수동 읽기 실시간 점수화
실질적 이해력	Outcome Metric	인터랙티브 퀴즈: 난이도 × 정답률 → '텍스트 복잡도 대비 이해도 (Comprehension Score)'

4.3 신뢰성 근거 (추가 질문 대비 키워드)

- **Flesch-Kincaid(한국어 대응) 가독성 지수**: 원문 난이도를 객관 정의하고 점수를 보정(Normalization)해 공정 비교.
- **Cross-Validation**: 퀴즈 정답 + 스크롤 패턴 교차 검증으로 '찍기' 배제, 데이터 신뢰도 향상.
- **Longitudinal Study**: 수주간 누적 시계열로 추세 분석 → 일시적 오류 배제.

❌ 절대 하지 말아야 할 말 (과장 = 거짓)

- "의사·전문가의 임상 시험을 거쳤습니다" — 실제로 안 했으므로 거짓.
- "뇌파(EEG)를 측정합니다" — 하드웨어 없음, 불가능.
- "사용자가 좋아하니까 향상된 겁니다" — 주관적 느낌은 과학적 근거 아님.

핵심 요약 답변: "우리는 읽는 과정(행동 데이터)과 읽은 결과(퀴즈 데이터)를 결합해 인지부하와 이해도를 정량화함으로써 문해력 향상을 과학적으로 추적합니다."

LangGraph 기반 **계층형 멀티 에이전트**. 메인 오케스트레이터가 전문 서브 에이전트를 동적으로 제어하고 결과물을 실시간 검증·교정한다.



5.1 동적 호출 메커니즘 (StateGraph)

사용자 상태 변화에 따라 에이전트 간 전이(Transition) 발생:

Content Reducer → Cognitive Care → Reward Agent → Literacy Profile Agent 순의 유기적 호출, 오케스트레이터가 공유 상태(Shared State) 기반으로 라우팅.

5.2 핵심 설계 원칙 — 입력원 독립

오케스트레이터는 **이벤트의 출처를 가리지 않는다**. 입력은 `reading_events` (scroll/pause/blur/focus) + 글 텍스트, 출력은 개입 명령(넛지·하이라이트)과 세션 결과(score)뿐이다. 이 이벤트가 **웹앱에서 모든 크롬 확장에서 모든 PDF 뷰어에서 모든 동일하다**. → 확장 추가는 "코어 교체"가 아니라 "새 입력원 추가"이며, 기존 계약·변환 어댑터를 거의 그대로 재사용한다. (상세 §9)

5.3 에이전트 색인

① Main Orchestrator

② Content Reducer

③ Cognitive Care

④ Reward

⑤ Literacy Profile

⑥ QA / Evaluation (신규)

1 Main Orchestrator Agent

Orchestration & QA Loop

역할: 전체 에이전트 제어·상태(State) 관리 / 동적 워크플로우 / 결과물 검증·품질관리(Self-Correction Loop) / Shared State 기반 협업·라우팅.

기술: LangGraph(상태·워크플로우), StateGraph(공유상태 전이), FastAPI(비동기 이벤트), Python Rule-Based(적합성 판정), Redis(세션 캐시), Hybrid LLM Routing.

처리 데이터: 문해력 프로필, 문서 컨텍스트, 실시간 집중도·진행률, 퀴즈 결과·행동 로그, 재작업 로그 (Refinement Logs).

2 Content Reducer Agent

Readability & Restructure

역할: 긴 글의 구조 분석·의미 단위 분할 / 사용자 수준별 가독성 최적화·재구성.

기술: LLM(Claude/GPT-4o, 고난도 재구성), LangChain SemanticChunker, Flesch-Kincaid 유사 알고리즘, Prompt Engineering, Adaptive Reading Logic, **RAG(용어풀이 그라운드링)**.

처리 과정: 원문 → 난이도 분석 → Semantic Chunking → 쉬운 문장 변환(Terminology Mapping) → 제공

입력원 정규화: 웹(Readability 추출) / PDF(pdf.js getTextContent) 두 소스를 **동일한** `content[]` 형태로 받는다.

3 Cognitive Care Agent

Focus & Adaptive Nudge

역할: 실시간 행동 분석 기반 집중도 측정 / 산만함 감지 시 적응형 네티지.

기술: FastAPI(이벤트 구동 수집), Redis(세션 로그), LLM(문맥 기반 퀴즈), Python Scoring Logic(다변량 집중도 점수).

수집 데이터: 스크롤 속도·패턴, 체류 시간, 페이지 이탈(Tab Blur), 클릭 패턴, 읽기 진행률.

Adaptive Nudge 3단계:

- Soft** 핵심 문장 하이라이트, 폰트·가독성 최적화
- Medium** "잠시 멈춰 다시 읽어볼까요?" 텍스트 가이드
- Hard** 즉석 퀴즈로 인지적 재집중(Re-engagement) 유도

4 Reward Agent

Gamification & Feedback

역할: 완독 데이터 분석 기반 즉각 보상·성취감 / 정량 피드백 생성.

기술: LLM(성장 메시지), PostgreSQL(완독·성취 영구 저장), Redis(실시간 경험치), Python Analytics(성장 지표).

산출물: 완독률·집중도·정답률 기반 경험치·배지 / 주간·월간 집중도 변화 & **문해력 성장 추이 시각화**.

5 Literacy Profile Agent

Personalization Engine

역할: 장기 데이터 추적 기반 인지 프로파일 구축 / 취약 영역 분석·개인화 학습경로.

기술: PostgreSQL(고차원 프로파일), Python Analytics(장기 패턴·상관 분석), LLM(콘텐츠·경로 추천).

분석(Deep Insights): 인지적 취약점(집중 저하 주제·문장구조), 언어적 장벽(반복 인지부하 용어), 행동 특성(선호 속도·최적 집중 시간대).

최종 결과물: Dynamic Literacy Profile + Personalized Learning Path → **Content Reducer 난이도 조정**에 재반영(피드백 루프).

6 QA / Evaluation Agent

신규 — Product Stability

추가 배경: "소프트웨어 필수는 기획·구현·QA(테스트)" — QA를 별도로 분리.

역할: 제품 동작 안정성 검증(런타임 결과 품질과 별개 층). 정상/이상 동작·버그 탐지, 단위·통합 테스트 자동화.

기술: Promptfoo·Ragas 스위트를 **CI처럼 자동 회귀 검증**, LangSmith 추적 연동.

포인트: 사용자 노출 런타임 에이전트가 아닌 **엔지니어링 검증 에이전트** — "검증 체계까지 설계한 팀"이라는 성숙도 가점.

7.1 RAG는 "딱 한 곳"에만

RAG를 전역에 뿌리면 **버즈워드 남발**로 보인다. **Content Reducer**의 **용어풀이/쉬운 설명**에만 한정한다.

항목	내용
적용 위치	어려운 전문용어 풀이 · 쉬운 설명 (Content Reducer)
그라운드링 출처	신뢰 출처(표준국어대사전·도메인 용어집·위키 등)
효과	환각 방지 → Ragas Faithfulness 로 "GPT처럼 지어내지 않음"을 정량 증명 → 차별화 강화

7.2 공통 인프라

계층	스택
Backend	Python 3.11, FastAPI, REST(event-driven), Async Processing
Database	PostgreSQL(관계형 데이터), Redis(실시간 상태)
Agent Framework	LangGraph, StateGraph, LangChain, SemanticChunker, Hybrid LLM Routing
Extension	Manifest V3, declarativeNetRequest, pdf.js(Apache-2.0 번들), Shadow DOM

7.3 Hybrid LLM Routing — 모델 선택 확정

경로	담당 작업	모델 (확정/권장)
High-end	고난도 추론, 복잡한 문장 재구성, 프로필 분석	Claude (Opus / Sonnet) 1순위 — 한국어 문해·충실 재작성 강점
Light-weight	단순 청킹, 기초 점수 계산, 단순 문장 변환	경량 모델 (비용·레이턴시 최적화)

→ "한국어 품질 + 비용 효율"을 동시에 고려했다는 설계 근거가 됨. 데모/개발은 **stub 경로**로 과금 없이 돌린다(\$9-9).

8

검증·평가 체계 (2층 구조)

Harness Engineering + Product QA

핵심 정리: "오케스트레이터가 이미 검증하는데 QA를 또?"라는 의문의 답 — 층(Layer)이 다르기 때문이다.

구분	A층 — 런타임 결과 품질	B층 — 제품 동작 안정성 (신규)
검사 대상	에이전트가 내놓은 답이 좋은가	서비스가 실제로 잘 굴러가는가
담당	Orchestrator Self-Correction + Ragas	QA / Evaluation Agent
방식	요약 충실도·퀴즈 적합성 실시간 평가	단위/통합 테스트, 회귀, 버그 탐지(CI)
시점	사용자 요청 처리 중(런타임)	빌드·배포 시(개발 파이프라인)

8.1 Harness Engineering 도구

도구	역할
Promptfoo	프롬프트 회귀 테스트 및 성능 저하 검증
Ragas	요약 품질(Faithfulness) 및 퀴즈 적합성(Relevance) 평가
LangSmith	에이전트 호출 추적(Tracing), 토큰 비용·레이턴시 모니터링
pytest	코어 페루프·계약·라우팅·점수 엔진 단위/통합 테스트(결정적 검증)

8.2 핵심 평가 지표

- Faithfulness
- Answer Relevance
- Readability Score
- Latency
- Token Cost
- User Engagement
- Completion Rate

발표 포인트: "기획 → 구현 → QA"의 SW 개발 정석을 갖추고, 응답 신뢰성·요약 충실도·실시간 데이터 정밀함을 이중 검증하는 하네스 엔지니어링 체계를 도입.

9.1 제품 형태 — 파일 업로드 없는 크롬 확장(주력)

웹 서핑 중 접하는 모든 텍스트(뉴스·블로그·논문·기사·PDF)에 실시간 개입하는 **브라우저 확장 프로그램**.
on/off 토글로 켜두면 **백그라운드에서 상시 동작**하여, 사용자가 "보던 대로" 읽는 동안 측정·개입·점수화한다.

컴포넌트	역할	설명
크롬 확장	주력	실제 읽기 측정 + 라이브 넷지 오버레이. 웹페이지 + PDF(pdf.js 뷰어) 모두 지원
웹 대시보드	공용	점수/성장/배지 확인 — 성장 그래프(데모 핵심 화면)
웹 읽기 화면	폴백	무설치 데모용 — 심사위원이 확장 미설치 시 시연

읽기 UX를 양쪽에서 중복 유지하지 않는다: 확장=주력 / 웹읽기=데모 폴백 / 대시보드=공용.

9.2 확장 아키텍처 (Manifest V3)

```
# extension/
├─ manifest.json      declarativeNetRequest + web_accessible_resources
├─ popup/             on/off 토글 · 온보딩(동의) · 파일 열기
├─ background/        service worker: 세션 수명 · PDF 리다이렉트 규칙
├─ content/           웹페이지 주입(shared 어댑터)
├─ pdf/               pdf.js 자체 뷰어(viewer.html · js · css)
├─ vendor/pdfjs/      pdf.mjs · pdf.worker.mjs (Apache-2.0 번들)
├─ shared/            # 웹 · PDF 공용화(중복 제거)
│   ├── tracker.js    읽기행동 이벤트 캡처(정규화 스키마 방출)
│   ├── overlay.js    Shadow DOM 오버레이(넷지 · 배지 · 퀴즈 · 톨팁)
│   └── session_client.js 세션 수명 + REST 배치 flush 전송
```

MV3 현실: service worker는 상시 데몬이 아니라 **이벤트로 자고 깬다**. 탭/네비게이션 이벤트마다 깨어나 모니터링 → 체감은 "항상 켜짐". 상태는 `chrome.storage` 에 두어 진짜 24/7 프로세스는 불필요.

9.3 확장 ↔ 백엔드 계약 (기존 계약 재사용)

엔드포인트	역할
POST /api/session/start	추출 본문 <code>content[]</code> 로 세션 시작(웹=Readability / PDF=pdf.js). 응답 camelCase
POST /api/session/{id}/events	읽기행동 이벤트 배치 flush → 응답에 실린 개입 렌더(REST 이벤트 구동)
GET /api/session/{id}/result	전체 오케스트레이터 실행 → Literacy Score·배지·시계열 반환

→ 백엔드 관점에선 "웹이든 PDF든 `content[]` 받으면 끝" — **신규 계약 불필요**. 전송방식은 REST로 확정(ADR-001, §9-8).

9.4 왜 기본 뷰어로는 안 되고, 왜 pdf.js면 되는가

크롬 기본 PDF뷰어는 PDFium(네이티브 플러그인)이 페이지를 "그림처럼" 그린다 → 확장이 **글자·좌표·스크롤에 접근 불가** = 스크롤 속도·퀴즈·단어 뜻 전부 불가.

pdf.js(Mozilla)는 PDF를 **JavaScript로 다시 그려 DOM으로 되돌린다**. 페이지를 `<canvas>` 로 그리고 그 위에 **단어마다** `<span>` **텍스트 레이어**를 좌표 맞춰 깎다 → PDF가 "일반 웹페이지"가 되어 **웹과 똑같은** 트래커/오버레이를 그대로 쓴다.

핵심: 사용자 경험은 기본 뷰어와 동일(PDF 링크 클릭 → 열림, 업로드 없음). 바뀌는 건 "누가 렌더하느냐"뿐 — 크롬 대신 우리가 렌더해 만질 수 있게 만든다.

읽는 방식	스크롤 속도	창 위 퀴즈	단어 뜻	사용자 경험
크롬 웹페이지	✓	✓	✓	그대로
크롬 기본 PDF뷰어	✗	✗	✗	그대로
우리 pdf.js 뷰어	✓	✓	✓	그대로(자동 전환)

9.5 동작 흐름

PDF 링크 클릭 (`https://...pdf` 또는 로컬 파일)

- ↳ declarativeNetRequest 규칙이 main_frame 요청을 가로챈
 - chrome-extension://<id>/pdf/viewer.html?file=<원본 URL>
 - ↳ pdf.js가 원본을 fetch → 페이지별 canvas + textLayer() 렌더
 - ↳ getTextContent()로 본문 추출 → content[] 로 POST /session/start
 - ↳ tracker(공용): 스크롤 속도 · 체류 · blur/focus → 백엔드
 - ↳ overlay(공용): 넋지 · 퀴즈 · 단어 뜻 톨팁
 - ↳ 단어 hover → textLayer span에서 단어 추출 → 용어풀이 → 톨팁

로컬 PDF는 **파일 피커** → **ArrayBuffer**로 브라우저 로컬 렌더(서버 저장 없음). 스캔(이미지) PDF는 글자 레이어가 없어 MVP 제외 → OCR(Tesseract.js·무료)은 후속.

9.6 세션 수명 (on/off + 백그라운드)

- [확장 off] → 아무 동작 안 함
- [확장 on] → 페이지 로드 → content script 주입
 - "읽을 만한 글" 판정? — 예 → N초 체류/스크롤 → 세션 시작 + 이벤트 스트리밍
 - 실시간 focus 하락 → 넋지 오버레이
 - 탭 이탈/닫기/visibility hidden 지속 → 세션 종료 → result → 대시보드 기록

9.7 온보딩 & 사용자 식별 (비용 0 · PII 0)

항목	내용
사용자 식별	설치 시 익명 UUID (<code>crypto.randomUUID()</code> → <code>chrome.storage</code>). 로그인·회원가입 없음 . 익명 ID로도 프로필 누적 성립
문서 열기	로컬 PDF를 파일 피커 → pdf.js 뷰어로 렌더. 서버 저장 없음 (ArrayBuffer 로컬 처리)
온보딩	확장 팝업 최초 1회: 개인정보 동의 + ON/OFF(기본 OFF). 동의 전에는 아무 것도 수집 안 함
정직 고지	"안 하는 것" 명시 — 화면 상시 감시 아님(ON+읽을만한 글일 때만) · EEG/카메라 없음 · 크롬 밖 앱 안 봄

이 "안 하는 것" 고지는 §4.3의 "❌ 하지 말아야 할 말"과 같은 정직 원칙 → **심사 방어**로 직결.

9.8 UI 인터페이스

Adaptive Reading Interface (Shadow DOM 오버레이) • Contextual Tooltips: 용어 hover 시 쉬운 풀이 팝업 (RAG 그라운드) • Smart Highlighting: 핵심 문장 자동 볼드·배경 강조 • Interruption Pop-up: 집중 저하 시 흐름을 깨지 않는 퀴즈 카드	Growth Dashboard + Floating Panel • Floating Control Panel: 집중도·진행률·실시간 경험치 미니 대시보드 • Visualized Analytics: 문해력 성장 그래프·주간/월간 통계 ★데모 핵심 • Gamified Profile: 독서 레벨·배지·연속 완독 기록
--	---

9.9 설계 결정 기록 (ADR) & 비용 0 원칙

결정	내용 & 근거
ADR-001 전송 방식	확장↔백엔드 이벤트/가입 왕복 = REST(event-driven) . 가입이 전부 행동 반응형이라 서버 push 불요 + 백엔드 REST 이미 구현·테스트 완료 → 신규작업 0. WS는 후속(계약이 전송무관이라 저비용 전환)
ADR-002 식별·문서	익명 기기 UUID + 로컬 pdf.js 문서 처리 + 팝업 온보딩. 마찰 0·비용 0·PII 없음. 구글 OAuth는 기기간 동기화 필요 시 후속(이메일/비번 자체구현·서버 업로드 보관은 기각)
비용 0 원칙	새로 과금되는 요소 없음. pdf.js(Apache-2.0)·Tesseract.js(후속, Apache-2.0) 무료 번들 , 유료 API/호스팅 금지, 데모는 stub로 무과금 . LLM 호출이 유료라면 그건 기존 오케스트레이터 이슈이지 확장이 새로 만든 비용이 아님

범위 밖(후속 트랙): "크롬 밖 다른 뷰어·모든 앱 창" 측정은 네이티브 데스크톱 에이전트 = 별도 트랙.

10 개발 우선순위 & 할 일 정리

Priority & Task Backlog (시간 무관)

일정형 데일리 플랜 대신 **"무엇을 할지"**만 정리한다. 아래 우선순위로 하되, 각 역할은 자기 표의 **예정** 항목을 위에서부터 처리하면 된다. **완료**=구현·검증됨 / **진행**=일부 됨 / **예정**=미착수.

10.1 시연 우선순위

우선	범위	이유
필수	한 편의 글 → 실시간 집중도 측정 → 넋지/퀴즈 → Literacy Score 전후 그 래프	이 페루프 하나면 GPT 차별화가 증명됨
필수	크롬 확장으로 웹 + PDF 실측정 (업로드 없이 "보던 대로")	"ChatGPT에 붙여넣기"와의 결정적 사용성 차이 시연
권장	Content Reducer(쉬운 변환 + RAG 용어풀이), 게이미피케이션(배지·레벨)	경험 완성도·재방문 동기
권장	QA / Eval Agent 자동 검증 데모	엔지니어링 성숙도 가점
후순위	스캔 PDF OCR, 기기간 동기화 (OAuth), 네이티브 데스크톱 에이전트	비전으로 제시, 데모 범위 밖

10.2 마일스톤 게이트 (완료 기준, 날짜 무관)

게이트	완료 기준(Done When)
M0	더미 데이터로 E2E 통신 OK (raw_text 입력 → 결과 JSON 반환) 완료
M1	핵심 페루프 데모 동작 (측정 → 개입 → 점수) 완료
M2	전 기능 통합 — 실제 팀원 모듈 일부 이상 연결 진행
M3	기능 동결 — 구조 변경 없이 버그 수정만 예정
M4 / 제출	제출본 점검 — 시연 흐름 반복 실행 가능 예정

10

역할별 할 일 (시간 무관)

Tasks by Role — 무엇을 할지

각 역할은 자기 표의 **예정** 항목을 우선순위로 처리. 상태는 현재 시점 기준.

① 에이전트 코어 / 오케스트레이션

기술리드(이소희)

할 일	상태
Shared State 스키마 + 에이전트 입출력 계약(API_CONTRACT) 확정	완료
Orchestrator flow + 조건부 라우팅 + Self-Correction/Fallback/Trace	완료
Literacy Score 합성 엔진(순수 함수·결정적)	완료
stub 기반 E2E + stub/real 토글 구조	완료
확장 인입 계약: <code>/session/start(content[])</code> · <code>/events (REST)</code> · <code>/result</code>	완료
웹·PDF 공용 tracker/overlay 계약 + 세션 경계 정의	완료
consent/userId(익명) 상태 스키마 + 백엔드 userId 계약 정합	완료
2번 Content Reducer real 연결 (stub→실모듈 글루)	예정
시계열 추세 산출 마무리 · 점수 보정(가독성·교차검증) 튜닝	예정
M2 전 기능 통합 디버깅 · origin/team 레포 동기화	예정

② 콘텐츠 & RAG 에이전트

할 일	상태
Semantic Chunking + 쉬운 문장 변환 파이프라인	예정
난이도 분석(한국어 가독성 지수) 연동	예정
RAG 용어풀이 그라운드(무료 경로만) + Faithfulness 검증	예정
pdf.js <code>getTextContent()</code> → 문단 재구성(줄병합·하이픈·머리말/꼬리말 제거)	예정
웹(Readability)/PDF(pdf.js) 두 소스 → 동일 <code>content[]</code> 정규화	예정
문맥 기반 인터랙티브 퀴즈 프롬프트 + Reward 성장 메시지	예정

③ 백엔드 & 실시간 데이터

할 일	상태
FastAPI 세션 스토어 + /events 개입 동기 반환(REST)	완료
CORS(chrome-extension:// .pdf 뷰어 오리진) 허용	완료
익명 userId 수용(기본값 anonymous)	완료
집중도 Scoring 실데이터 연동 · 행동로그 저장	예정
시계열 누적 저장 + Literacy Profile 적재(PostgreSQL/Redis)	예정
단어 뜻 lookup 응답(무료 로컬/기존 에이전트) · (후속) 데이터 삭제 요청	예정

④ 프론트엔드 & 확장 UI

할 일	상태
확장 MVP 골격(manifest·background·content·popup ON/OFF)	완료
pdf.js 뷰어 렌더(canvas + textLayer) + shared 모듈 연결	진행
팝업 온보딩 UI (개인정보 동의 + "안 하는 것" 고지 + 파일 피커 + UUID)	예정
단어 hover 툴팁 UI + 퀴즈 모달 (overlay 확장)	예정
성장 대시보드(주간·월간 그래프) + 게이미피케이션(배지·레벨) + Floating Panel	예정
확장 클라이언트 WS → 배치 flush POST 전환(ADR-001)	예정

⑤ QA / 평가 & 통합

할 일	상태
pytest 코어 페루프·계약·점수 단위/통합 테스트	완료
Ragas-Promptfoo 파이프라인(Faithfulness/Relevance) + LangSmith 추적	예정
확장 수동 브라우저 E2E (로드 → ON → 기사/PDF 왕복 · 오버레이·넛지 확인)	예정
통합 스모크/회귀 + 시연 리허설 + 배포·데모 환경 구성	예정

협업 규칙: ①이 계약(공유 상태 스키마)을 먼저 확정 → 각 파트 병렬 개발. 필수 페루프는 ①·③·④가 먼저 "측정 → 개입 → 그래프"를 완성하고, ②·⑤가 뒤이어 합류한다.

11 개발 단계 역할 분담

Development Roles (5)

기획은 확정 완료 — 아래는 **개발 단계** 기준 5인 분담. (기술리드: 이소희 / 컴퓨터과학과)

개발 파트	담당 모듈 · 에이전트	핵심 개발 작업	산출물
① 에이전트 코어 / 오케스트레이션 (기술리드)	Main Orchestrator(①) · Hybrid LLM Routing · StateGraph · 확장 계약/클루	LangGraph 워크플로우·공유상태 설계, 에이전트 전이·라우팅, Self-Correction, Literacy Score 엔진 총괄 , 확장↔백엔드 계약·세션 수명	멀티에이전트 코어, 점수 엔진, 확장 인입 계약
② 콘텐츠 & RAG 에이전트	Content Reducer(②) · 본문 추출 · Reward 메시지 일부(④)	Semantic Chunking, 난이도 분석, 쉬운 문장 변환, RAG 용어풀이 그래운딩 , pdf.js/Readability → 동일 content[] 정규화	재구성 파이프라인, RAG 용어풀이 모듈
③ 백엔드 & 실시간 데이터	Cognitive Care 백엔드(③) · Literacy Profile 적재(⑤) · 인프라 · CORS	FastAPI REST 이벤트 수집, 집중도 Scoring Logic , PostgreSQL/Redis 스키마, 시계열 누적, 익명 userId-CORS	데이터 파이프라인, 집중도 측정 API
④ 프론트엔드 & 확장 UI	Overlay · Floating Panel · pdf.js 뷰어 UI · 팝업 온보딩 · Reward 시각화(④)	하이라이트·용어 톨팁·퀴즈 모달, 팝업 동의/토글, 문해력 성장 그래프(데모 핵심) , 게이미피케이션	확장 데모, 성장 대시보드
⑤ QA / 평가 & 통합	QA · Evaluation Agent(⑥) · Harness · 확장 E2E	Promptfoo·Ragas·LangSmith, 단위/통합 테스트, 확장 수동 브라우저 E2E , 빌드·배포·시연 환경	테스트 스위트, 평가 리포트, 배포 환경

협업 규칙: 착수 전 ①이 에이전트 간 API 계약(공유 상태 스키마)을 먼저 확정 → 각 파트 병렬 개발. **필수 데모(페루프)**는 ①·③·④가 우선 협업해 "측정 → 개입 → Literacy Score 그래프"를 먼저 완성하고, ②·⑤가 뒤이어 합류한다.

12 심사 예상 질문 & 방어

Q&A Defense

Q1. ChatGPT에 PDF 넣고 요약시키면 끝 아닌가요?

▶ GPT는 한 번 읽고 잊습니다. 우리는 어떻게 읽는지 측정하고, 이해했는지 검증하고, 나아지는지 추적하는 페루프 시스템입니다. 게다가 붙여넣기 없이 크롬에서 보던 대로 측정합니다.

Q2. 문해력 향상을 정말 측정할 수 있나요? (임상적으로)

▶ 읽는 과정(행동 데이터)과 읽은 결과(퀴즈 데이터)를 결합해 인지부하·이해도를 정량화하고, 시계열 추세로 성장을 추적합니다. (EEG·임상시험 주장 금지)

Q3. 그 점수를 신뢰할 수 있나요?

- ▶ Flesch-Kincaid 가독성 보정으로 공정 비교, Cross-Validation으로 '찍기' 배제, Longitudinal로 일시 오류 배제. 점수 엔진은 LLM이 아닌 결정적 순수 함수라 재현됩니다.

Q4. 크롬 기본 뷰어는 PDF를 못 만진다던데, PDF는 어떻게 측정하나요?

- ▶ 기본 뷰어(PDFium)는 그림처럼 그려 접근 불가라, PDF 링크를 우리 pdf.js 뷰어로 자동 전환해 DOM으로 되돌립니다. 그러면 웹페이지와 똑같이 스크롤·퀴즈·단어 뜻이 동작합니다. 사용자 경험은 동일합니다.

Q5. 개인정보·로그인은요? 상시 감시 아닌가요?

- ▶ 로그인 없이 익명 기기 UUID만 씁니다. ON일 때 읽을 만한 글에서만 동작하고, 화면 상시 감시·카메라·크롬 밖 앱은 보지 않습니다. 문서는 로컬 처리(서버 저장 없음)입니다.

Q6. 환각(hallucination)은 어떻게 막나요?

- ▶ 용어풀이는 신뢰 출처에 RAG 그라운드링하고, Ragas Faithfulness로 정량 검증합니다.

13.1 리스크 & 대응

리스크	대응
"GPT와 차별점 불명확" (최대 리스크)	페루프 프레임 + Literacy Score 전후 그래프로 시각 증명
범위 과다 → 미완성 데모	필수 페루프 우선, 나머지는 비전으로 제시
측정 점수 신뢰성 의심	가독성 보정·교차검증·시계열 3중 근거 + 결정적 순수함수
스캔(이미지) PDF엔 글자 레이어 없음	MVP는 "글자 레이어 있는 PDF"만. OCR(Tesseract.js·무료)은 후속
페이지 CSS와 오버레이 충돌	Shadow DOM 격리
PDF 리다이렉트 오작동(다운로드 링크 등)	main_frame·.pdf 만 조건화 + on/off·사이트 제외 규칙
확장 설치·심사 장벽(데모)	웹 읽기 화면을 무설치 폴백으로 유지
MV3 service worker 수명	이벤트 기반 설계, 상태는 chrome.storage에
LLM 비용·레이턴시	Hybrid Routing(고난도만 고성능), 데모는 stub 무과금, LangSmith 모니터링

13.2 기대효과 요약

1. **문해력 위기 실질 개입** — 읽기 과정 전체를 관리하는 통합 AI 리딩 코치.
2. **숫품 집중력 저하 해결** — 3단계 개입으로 2030 세대 문해력 상승.
3. **난독증·ADHD 배리어프리** — 한국어 AI 주석 기반(LARF 연구 +18.3% 근거).
4. **마찰 0 진입** — 업로드·로그인 없이 크롬에서 "보던 대로" 측정(웹 + PDF).
5. **게임화 장기 습관 형성** — 재방문 동기 구조로 실질적 향상.
6. **개인화 성장 경로** — 쓸수록 정교해지는 Dynamic Literacy Profile.

13.3 용어 정리

용어	의미
Literacy Score	이해도·집중도·난이도 보정을 합성한 본 서비스 핵심 문해력 지표(결정적 순수함수)
Closed-Loop	측정 → 개입 → 추적 → (재개인화) 순환 구조
Adaptive Nudge	집중 저하 시 단계별(Soft/Medium/Hard) 개입

pdf.js 자체 뷰어	PDF를 JS로 렌더해 DOM(canvas + textLayer)으로 되돌려 웹과 동일 측정을 가능케 하는 뷰어
declarativeNetRequest	MV3 API — PDF 링크 요청을 우리 뷰어로 리다이렉트하는 데 사용
Harness Engineering	Promptfoo·Ragas·LangSmith 기반 에이전트 품질 검증 체계
Dynamic Literacy Profile	누적 데이터로 실시간 갱신되는 사용자 인지 프로필